

Trabalho Prático

Bernardo Zschaber Morato Nogueira - 2024066121

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte - MG - Brasil

bernardozschaber@ufmg.br

1. Introdução

Este relatório descreve o desenvolvimento de duas soluções em linguagem Assembly para a arquitetura RISC-V, utilizando o simulador Venus (<https://venus.kvakil.me/>). O objetivo do trabalho foi aplicar e aprofundar os conhecimentos em programação de baixo nível, com foco em manipulação de memória, implementação de algoritmos e, crucialmente, o gerenciamento da pilha de chamadas de procedimentos.

O primeiro problema aborda a implementação do algoritmo de criptografia One-Time Pad (OTP), uma técnica de criptografia de fluxo que realiza uma operação XOR entre uma mensagem e uma chave secreta. O segundo problema consiste na implementação da multiplicação de matrizes quadradas, uma operação fundamental em áreas como processamento de imagens e computação gráfica, exigindo a criação de procedimentos aninhados e o correto uso da pilha. A documentação elaborada está dividida nos seguintes tópicos:

1. Introdução	1
2. Decisões de implementação tomadas	1
3. Dificuldades enfrentadas	2
4. Testes	2
5. Conclusão	2
6. Referências bibliográficas	2

2. Decisões de implementação tomadas

As soluções foram desenvolvidas seguindo as convenções da ISA RISC-V e as boas práticas de programação em Assembly.

Problema 1: One-Time Pad (onetime_pad)

Estrutura de Repetição: foi implementado um laço (loop) simples para iterar sobre os vetores de mensagem e chave. Um registrador temporário (t0) foi utilizado como contador do índice (i), começando em 0 e incrementando a cada iteração até atingir o tamanho do vetor, fornecido como argumento em a2.

Cálculo de Endereços: para acessar os elementos mensagem[i] e chave[i], foi necessário calcular o deslocamento (offset) em bytes a partir dos endereços base. Como cada elemento é uma palavra (.word), que ocupa 4 bytes, o deslocamento foi calculado como $i * 4$. A instrução slli (Shift Left Logical Immediate) foi escolhida por ser uma forma eficiente de realizar a multiplicação por 4 (deslocando 2 bits para a esquerda).

Operação lógica (XOR): os valores são carregados em t4 e t5 usando lw, e o resultado da operação XOR é obtido com xor t6, t4, t5. Esse valor é gravado de volta na própria mensagem (sw t6, 0(t2)), modificando o vetor original conforme especificado pelo algoritmo One-Time Pad.

Problema 2: Multiplicação de Matrizes (mat_mult e dot_product)

Decomposição do Problema: conforme solicitado, a lógica foi dividida em dois procedimentos:

mat_mult: responsável pela estrutura de alto nível, com dois laços aninhados para iterar sobre as linhas (i) e colunas (j) da matriz resultante C. *dot_product*: função auxiliar que calcula o produto escalar entre uma linha de A e uma coluna de B, recebendo como parâmetro adicional (a3) o índice da coluna desejada, e sendo chamada repetidamente por *mat_mult* para preencher cada elemento C[i][j].

Gerenciamento da Pilha (Stack): como requisitado, *mat_mult* é uma "função não-folha" (non-leaf function) — pois chama *dot_product*. No início de *mat_mult*, o ponteiro da pilha (sp) é decrementado para alocar espaço. Em seguida, o endereço de retorno (ra) e todos os registradores salvos (s1 a s6) que seriam utilizados para armazenar os contadores (i, j) e os argumentos da função (endereços das matrizes e dimensão n) são salvos na pilha. Ao final do procedimento, os valores são restaurados da pilha na ordem inversa e o ponteiro da pilha é liberado. A mesma lógica foi aplicada em *dot_product* para salvar os registradores s que ele utiliza. Essa abordagem garante que os valores não sejam corrompidos pela chamada aninhada para que retorne corretamente.

Cálculo de Endereços de Matrizes: as matrizes são armazenadas em um vetor contínuo (row-major). O acesso a um elemento M[i][j] de uma matriz n x n foi calculado pela fórmula: endereço base + ((i * n) + j) * 4. Esta lógica foi implementada usando instruções de multiplicação (mul), adição (add) e deslocamento de bits (slli) para determinar a posição exata de cada elemento na memória.

3. Dificuldades enfrentadas

Falha inicial no teste do dot_product: o cálculo do produto escalar estava correto isoladamente, retornando o valor esperado (19). No entanto, o teste automático não incrementava `s0` na segunda verificação, indicando que o resultado não era reconhecido como correto. Essa inconsistência ocorreu devido à forma como `mat_mult` preparava os parâmetros para a chamada de `dot_product`, interferindo no uso de registradores e no endereçamento da memória. Embora `dot_product` estivesse correto por si só, sua execução dentro de `mat_mult` utilizava o endereço errado da coluna da matriz B.

Erro de endereçamento da coluna de B: o principal desafio foi identificar que o erro residia no cálculo do endereço da coluna `j` em `mat_mult`. Inicialmente, o código passava apenas a base de B (la `a1`, `matriz_b`) e deixava que o cálculo da posição da coluna fosse feito dentro de `dot_product`. Contudo, o valor de `a3` (índice da coluna) estava sendo tratado de forma incorreta, resultando na leitura de elementos errados de B. A solução consistiu em ajustar o cálculo do deslocamento da coluna, garantindo que, para cada elemento `C[i][j]`, fosse lida a coluna correta de B. Após essa correção, ambos os testes passaram com sucesso, refletido no incremento correto de `s0 = 2`.

Gerenciamento da pilha e registradores: outro ponto crítico foi o gerenciamento correto da pilha. Inicialmente, esquecer de salvar algum registrador `s` ou o endereço de retorno `ra` antes da chamada `jal dot_product` causava comportamento imprevisível, como loops infinitos ou retornos incorretos. Foi necessário rastrear cuidadosamente cada valor salvo (`sw`) e restaurado (`lw`), garantindo que o ponteiro de pilha `sp` fosse corretamente ajustado. Essa prática assegurou que o estado do programa permanecesse consistente entre chamadas e que os resultados fossem confiáveis.

4. Testes

Os códigos foram validados utilizando os casos de teste fornecidos no esqueleto de cada problema, além de terem sido passados outros exaustivos não incluídos na solução apresentada, haja vista que não houve especificação no enunciado para que a submissão fosse feita, apenas para testar e garantir que funcionasse com valores diferentes dos pré-estabelecidos.

prob1.s: foram realizados dois testes. O primeiro com um vetor de 3 elementos (`mensagem1`) e o segundo com um de 4 elementos (`mensagem2`). Em ambos os casos, o procedimento `onetime_pad` modificou corretamente o vetor da mensagem, e as comparações com os valores esperados foram bem-sucedidas, fazendo com que o registrador `s0` terminasse com o valor 2.

prob2.s: também foram realizados dois testes principais previstos. O primeiro validou a função `mat_mult` completa, multiplicando duas matrizes 2x2 e comparando cada um dos 4 elementos da matriz resultante (`matriz_c`) com os valores esperados ([19, 22, 43, 50]). O segundo teste isolou e validou a função `dot_product`, confirmando que o produto escalar entre a primeira linha de A e a primeira coluna de B resultava em 19. Ambos os testes passaram com sucesso, resultando em `s0 = 2`.

5. Conclusão

Este trabalho prático foi extremamente valioso para solidificar conceitos fundamentais de arquitetura de computadores e programação de baixo nível. Os principais aprendizados foram:

Convenção de Chamada RISC-V: A distinção entre registradores temporários (`t`) e salvos (`s`) deixou de ser um conceito teórico e se tornou uma necessidade prática. Entender quando e por que salvar registradores na pilha é crucial para escrever código modular e correto.

Gerenciamento da Pilha: A implementação de chamadas de função aninhadas proporcionou uma compreensão profunda do funcionamento da pilha (`stack`) para salvar o contexto de execução (endereço de retorno, variáveis locais/salvas), permitindo que os procedimentos operem de forma independente.

Mapeamento de Estruturas de Dados: O trabalho forçou a pensar em como estruturas de dados de alto nível, como vetores e matrizes, são representadas e acessadas na memória linear de baixo nível, reforçando a importância do cálculo de endereços.

Eficiência em Baixo Nível: A escolha de instruções como `slli` em vez de `mul` para multiplicações por potências de 2 ilustra como o conhecimento da arquitetura pode levar a um código mais otimizado. Isso segue o princípio de projeto (2): “menor é mais rápido”.

6. Referências bibliográficas

Patterson, D. A., & Hennessy, J. L. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. Morgan Kaufmann, 2017.

RISC-V Foundation. *RISC-V Instruction Set Manual – Volume I: Unprivileged ISA*.

MORTENSEN, Morten Bop. *Ripes – Visual Computer Architecture Simulator: Introduction*. Disponível em: <https://github.com/mortbopet/Ripes/blob/master/docs/introduction.md>.

PARANAÍBA NETO, Omar. *Notas de aula da disciplina de Organização e Arquitetura de Computadores*.